Hidden Markov Models

How AI reasons about states it cannot directly observe



Learning goals

01 Explain why hidden states require probabilistic reasoning

02 Read transition and emission probability tables

03 Use evidence to update beliefs step by step

04 Understand filtering, prediction, smoothing, and Viterbi

05 Run and interpret a simple HMM in Python

Lecture roadmap

01 Why hidden states?

02 Markov chains

03 Building an HMM

04 Filtering & prediction

05 Smoothing & Viterbi

06 Python & practice

01 • Why hidden states?

A sensor reports a measurement, not the full truth about a system.

The central problem

HIDDEN CONDITION

A machine has an internal condition:

- Healthy
- Degrading
- Faulty

We cannot directly observe its true condition at every moment.

OBSERVABLE EVIDENCE

Sensors provide clues:

- Temperature
- Vibration
- Alarm status

Readings can be noisy. A high temperature does not prove a fault.

Hidden state versus observation

HIDDEN • WHAT WE WANT TO KNOW

OBSERVED • WHAT WE CAN MEASURE



The state influences the reading; the reading does not uniquely reveal the state.

Why a single alarm is not enough

- Sensors sometimes produce false alarms.
- A fault can exist before an alarm appears.
- The same reading can have different causes.
- Past observations can help interpret a new reading.

KEY IDEA

Use probabilities to combine noisy evidence with a model of how the system changes.

02 • Markov chains

Before hiding the states, understand how states change over time.

A Markov chain in one picture



A machine can stay in the same state or transition to another state.

A transition probability answers: “How likely is the next state?”

The Markov assumption

THE FORMULA

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots) = P(X_{t+1} \mid X_t)$$

"Tomorrow depends on today" is a useful first approximation.

WHAT EACH PART MEANS

X_t = state now

X_{t+1} = state at the next step

The current state summarizes the history needed for predicting the next state.

Transition matrix: two-state example

	NEXT STATE: Healthy	NEXT STATE: Faulty
Healthy	0.90	0.10
Faulty	0.20	0.80

How to read it

Healthy → Faulty = 0.10

Faulty → Healthy = 0.20

Each row adds to 1.00.

One-step prediction

THE FORMULA

$$p_{t+1} = p_t \times T$$

[0.80, 0.20] × [[0.90, 0.10], [0.20, 0.80]]

WHAT EACH PART MEANS

p_t = today's state probabilities

T = transition matrix

p_{t+1} = tomorrow's probabilities

Multiply the row vector by the matrix.

Worked example: predict tomorrow

START

Healthy	80.0%
Faulty	20.0%

AFTER ONE STEP

Healthy	76.0%
Faulty	24.0%

P(Healthy next) = 0.80×0.90 + 0.20×0.20 = 0.76

Python: predict a future state

PYTHON • RUN IN GOOGLE COLAB

```
import numpy as np

T = np.array([[0.90, 0.10],
              [0.20, 0.80]])

p = np.array([0.80, 0.20])

next_p = p @ T
print(next_p)
# [0.76 0.24]
```

READ THE CODE

1. Store the transition probabilities in T.

2. Store today's belief in p.

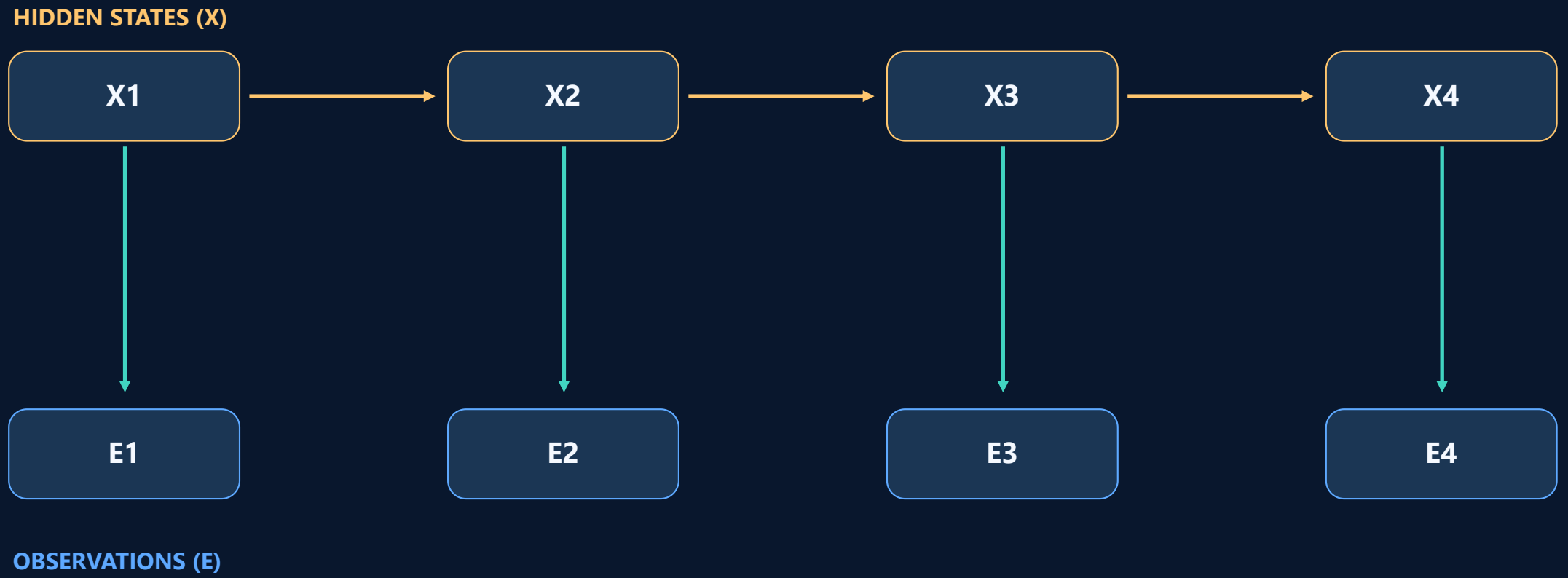
3. @ performs matrix multiplication.

4. Read the resulting probabilities.

03 • Building a Hidden Markov Model

Now the true state is hidden, but it produces observations.

HMM: two connected sequences



Horizontal: transitions • Vertical: emissions

The three ingredients of an HMM

1. Initial probabilities

π describes how likely each hidden state is at the beginning.

Example: 80% Healthy, 20% Faulty.

2. Transition probabilities

T describes how the hidden state changes between time steps.

Example: Healthy → Faulty = 10%.

3. Emission probabilities

O describes which readings are likely in each hidden state.

Example: Faulty → High = 70%.

Emission table: sensor readings

	SENSOR READING	
	Normal	High
Healthy	0.80	0.20
Faulty	0.30	0.70

Important distinction

P(High | Faulty) = 0.70

This is not the same as:

P(Faulty | High)

To reverse the condition, we must use Bayes' rule.

Reminder: Bayes' theorem

THE FORMULA

$$P(\text{State} \mid \text{Reading}) \propto P(\text{Reading} \mid \text{State}) \times P(\text{State})$$

Posterior = likelihood × prior ÷ total

WHAT EACH PART MEANS

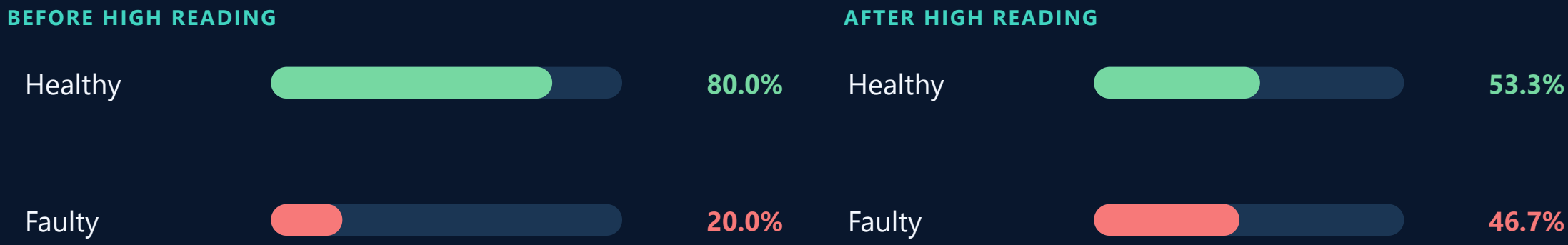
Prior: belief before a reading

Likelihood: how compatible the reading is with each state

Posterior: updated belief

Normalize so probabilities sum to 1.

One observation changes our belief



Healthy: 0.80×0.20 = 0.16 | Faulty: 0.20×0.70 = 0.14

Normalize by 0.16 + 0.14 = 0.30

04 • Filtering and prediction

Use a new reading to update what we believe about the current state.

Filtering: predict, observe, correct

1

PREDICT

Move the belief forward using T

→

2

OBSERVE

Receive a new sensor reading

→

3

CORRECT

Multiply by emission likelihood

→

4

NORMALIZE

Make the probabilities add to 1

Filtering estimates the current hidden state from observations received so far.

Filtering: the update equation

THE FORMULA

$$\text{belief}_t(j) \propto O_j(e_t) \times \Sigma_i \, \text{belief}_{t-1}(i) \, T_{ij}$$

REMINDER: prediction first, Bayes correction second.

WHAT EACH PART MEANS

Σ means “add over possible previous states.”

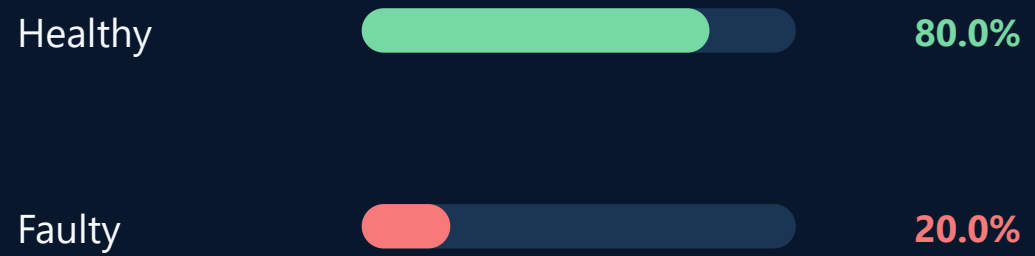
T_{ij} predicts a move from i to j.

$O_j(e_t)$ checks how likely the new reading is in state j.

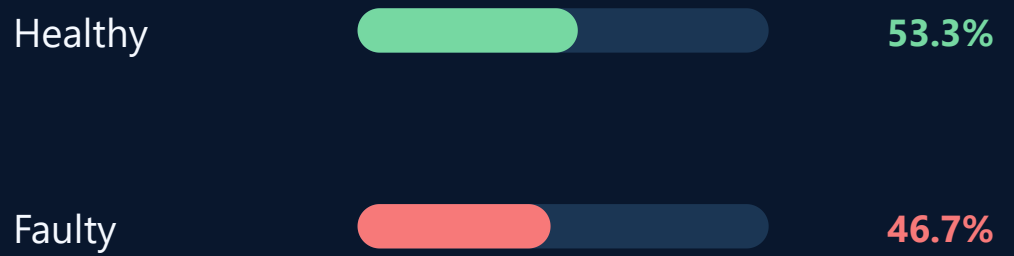
∝ means normalize at the end.

Filtering example: first reading is High

INITIAL BELIEF



AFTER HIGH



$0.80 \times 0.20 = 0.16$

$0.20 \times 0.70 = 0.14$

Normalize: [0.16, 0.14] ÷ 0.30 = [0.533, 0.467]

Filtering example: a second High reading

STEP A • PREDICT

$$[0.533, 0.467] \times T = [0.573, 0.427]$$

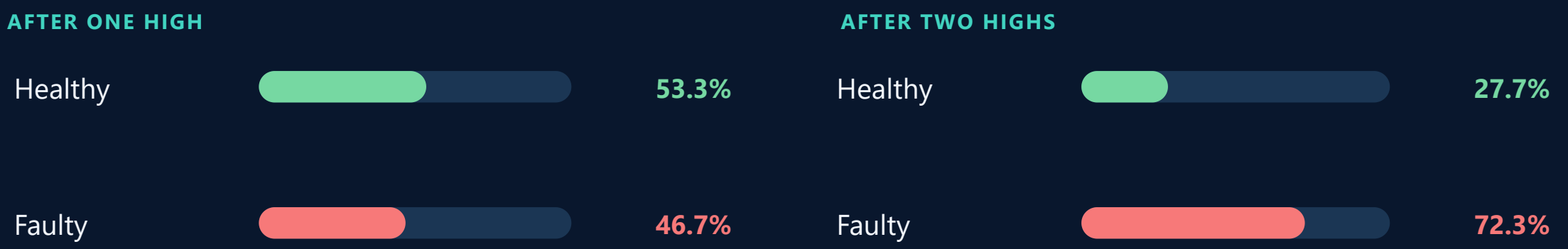
STEP B • CORRECT

$$[0.573 \times 0.20, 0.427 \times 0.70] = [0.115, 0.299]$$

STEP C • NORMALIZE

Posterior ≈ [0.277, 0.723] → Faulty is now more likely

Why two readings are more informative



Repeated evidence shifts the belief, while the transition model still matters.

Python: filtering two readings

PYTHON • RUN IN GOOGLE COLAB

```
import numpy as np
T = np.array([[.9,.1],[.2,.8]])
O = {'High': np.array([.2,.7])}
belief = np.array([.8,.2])

for t,reading in enumerate(['High','High']):
    if t > 0:
        belief = belief @ T
    belief *= O[reading]
    belief /= belief.sum()
    print(t+1, belief.round(3))
```

READ THE CODE

The first reading updates the initial belief.

For later readings, predict with T before correcting.

The emission likelihood weights each state.

Normalization produces a valid probability distribution.

Prediction after filtering

THE FORMULA

$$p_{t+k} = p_t \times T^k$$

For one step: $p_{t+1} = p_t \times T$

WHAT EACH PART MEANS

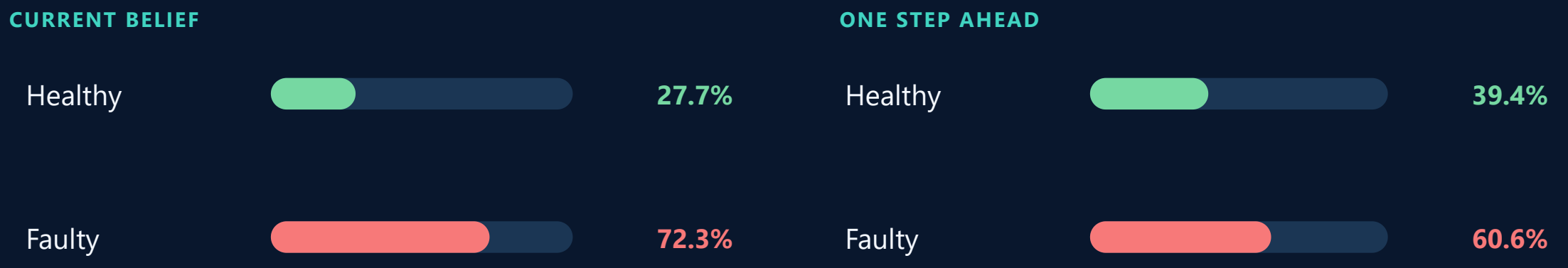
p_t = most recent filtered belief

T^k = k transition steps

No new readings are used during the prediction interval.

Prediction is about the future, not a new observation.

Forecast the next hour



Healthy next ≈ 0.277×0.90 + 0.723×0.20 = 0.394

05 • Smoothing and Viterbi

Two ways to learn about the past—without confusing their objectives.

Filtering versus smoothing

FILTERING

Uses observations collected up to the current time.

Question: “What is happening now?”

Example: Estimate the machine condition as each sensor reading arrives.

SMOOTHING

Uses both earlier and later observations to reconsider a past state.

Question: “What was happening then?”

Example: Reassess yesterday’s warning after today’s critical alarm.

Smoothing: later evidence changes our estimate



At time 2, we only know readings 1–2.

After time 4, readings 3–4 can improve our estimate of time 2.

Smoothing: conceptual formula

THE FORMULA

$$P(X_t \mid e_1,\ldots,eT) \propto \text{forward}_t \times \text{backward}_t$$

This is called the Forward–Backward method.

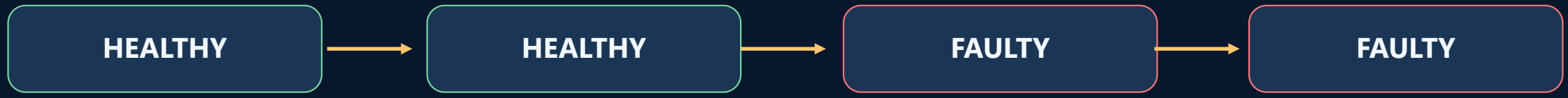
WHAT EACH PART MEANS

Forward message: evidence from the beginning through time t.

Backward message: how well later observations fit each possible state at t.

Multiply the messages and normalize.

Viterbi: one best hidden-state path



Which complete sequence is most probable?

Viterbi compares path probabilities and remembers the best predecessor.

Viterbi: remember the MAX

THE FORMULA

$$\delta_t(j) = O_j(e_t) \times \max_i[\delta_{t-1}(i)\, T_{ij}]$$

REMINDER: Filtering uses SUM; Viterbi uses MAX.

WHAT EACH PART MEANS

$\delta_t(j)$: score of the best path ending in state j.

max: keep only the most likely predecessor.

$O_j(e_t)$: include the current observation.

Store backpointers to recover the path.

Forward vs Viterbi: same model, different goal

FORWARD / FILTERING

Adds probability from all possible predecessor paths.

Operation: SUM

Output: state probabilities

Question: How likely is each state?

VITERBI / DECODING

Keeps the most likely predecessor for each state.

Operation: MAX

Output: one complete state sequence

Question: Which path is best?

Python: Viterbi with a library

PYTHON • RUN IN GOOGLE COLAB

```
# Install once in Google Colab:
# !pip install hmmlearn

from hmmlearn.hmm import CategoricalHMM
import numpy as np

model = CategoricalHMM(n_components=2,
    init_params='', params='')
model.startprob_ = np.array([.8,.2])
model.transmat_ = np.array([[.9,.1],[.2,.8]])
model.emissionprob_ = np.array([[.8,.2],[.3,.7]])
obs = np.array([[1],[1],[0]])
print(model.predict(obs))
```

READ THE CODE

Observation codes: 0 = Normal, 1 = High.

The model is given fixed probabilities.

model.predict() decodes the most likely hidden-state sequence using Viterbi by default.

State 0 = Healthy; State 1 = Faulty.

06 • Putting it all together

Build a clear mental model, then practice interpreting the results.

Which algorithm should you choose?

CURRENT STATE	**Filtering**	Use all readings received so far
FUTURE STATE	**Prediction**	Apply transitions without new readings
PAST STATE	**Smoothing**	Use both earlier and later readings
BEST PATH	**Viterbi**	Find the most likely entire sequence

Real-world use cases

Predictive maintenance

Hidden: machine condition

Observed: temperature, vibration, alarms

Goal: monitor deterioration

Speech recognition

Hidden: sound/phonetic states

Observed: acoustic features

Goal: decode a sequence

Robot localization

Hidden: robot position

Observed: noisy sensor signals

Goal: estimate location

Important limitations of a basic HMM

1. The hidden state is assumed to follow a first-order Markov process.
2. An observation depends on the current hidden state.
3. Probabilities must be estimated from data or supplied by experts.
4. Real systems may require richer models and careful validation.

Guided exercise: read the probabilities

GIVEN

P(Healthy) = 0.60
P(Faulty) = 0.40

P(High | Healthy) = 0.20
P(High | Faulty) = 0.70

Your task

A High reading arrives.

1. Compute both unnormalized scores.
2. Normalize them.
3. Which state is more likely?

Guided exercise: solution

Healthy score = 0.60 × 0.20 = 0.12

Faulty score = 0.40 × 0.70 = 0.28

Total = 0.12 + 0.28 = 0.40

P(Faulty | High) = 0.28 / 0.40 = 70%

Knowledge check

1. Why is an alarm not the same as a hidden state?

2. What does a row of the transition matrix represent?

3. Why do we normalize after using an observation?

4. How does smoothing differ from filtering?

5. What is the key difference between Forward and Viterbi?

Lecture summary: five things to remember

1	**HIDDEN**	The real state is not directly visible
2	**TRANSITIONS**	States change probabilistically
3	**EMISSIONS**	States generate noisy readings
4	**FILTERING**	Predict → correct → normalize
5	**VITERBI**	Find one best complete path

From hidden states to intelligent decisions



Hidden Markov Models turn uncertain time-series evidence into useful beliefs about the world.